Handoff 08: Nonce-based Content-Security-Policy (stage 8: SEC-09)

Date: 2026-10-26 (America/Toronto)
Written by: Claude Code (implementer)
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request, not merged, not deployed. The new policy is **Report-Only**: it blocks nothing yet. No content, XP or account changes.

> **How to use this file:** upload it with `build.md`, `docs/plan/backlog.md`, `docs/plan/feedback-security.md` and handoffs 06 and 07. It summarizes evidence; the pull request diff and CI results are the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 8 (SEC-09): the page
Content-Security-Policy now allows inline scripts only if they carry a
random nonce made for that one request, instead of 'unsafe-inline'. It
ships in Report-Only mode. I'm attaching build.md, the backlog, the
feedback security plan, handoffs 06 and 07, and this handoff. Check that
the nonce is unpredictable and new on every page request, that no page
can be served with a stale or shared nonce, that the Python worker's
policy is untouched, and that the one-line switch to enforce mode is
complete. Question the decision not to use 'strict-dynamic'. Explain at
a first-year computer engineering level.
```

1. Where we are

Stage	What	Status
1–6	Content, ledgers, catalog, goals, backlog, rate limits and headers	Verified live
7	SEC-03 server-side sessions	Merged (PR #13); your live check (handoff 07 section 10) not yet recorded
8	SEC-09 nonce-based page CSP	**in_review**: this PR

2. Handoff record (build.md template)

Field	Value
Task	SEC-09: replace `'unsafe-inline'` in the page policy's `script-src` with a per-request nonce; ship it Report-Only; explain the caching cost; tests; a one-line switch to enforce, tested locally
Human goal	An injected `<script>` (for example stored XSS in the future admin inbox) cannot run, because it cannot know the nonce
Repository and branch	`torontojohn/python-academy`, branch `sec/csp-nonces`
Base commit	`4848d49` (`main` after PR #13)
Implementer / reviewer	Claude Code / owner, plus ChatGPT or Claude chat
Deployment status	Not deployed; not merged
New dependencies	**None.** Uses Next.js's own proxy file and Web Crypto, which are built in

Files changed

File	Change
`frontend/proxy.ts`	New. Runs before every page request: makes a nonce, gives it to Next.js, and sends the policy. Has the `ENFORCE` switch
`frontend/csp.mjs`	New. The policies in one place: handoff 06's `PAGE_POLICY`, the new `noncePolicy(nonce)`, and the unchanged `WORKER_POLICY`
`frontend/next.config.mjs`	Imports `WORKER_POLICY`; no longer sends the page policy, because the proxy now does. The enforced `frame-ancestors 'none'`, HSTS and the other headers are unchanged
`frontend/app/layout.tsx`	`await connection()`, so every page renders per request and can receive its nonce
`frontend/app/not-found.tsx`	New. A plain 404 page using the app's stylesheet. Next.js's built-in 404 page uses inline styles that the policy blocks (section 4)
`frontend/tests/academy.spec.ts`	The header test now expects the nonce policy; three new tests (section 7)

File	Change
`DEPLOYMENT.md`, `docs/plan/backlog.md`, `docs/plan/feedback-security.md`, handoff 06	SEC-09 in_review; stage 7 merged; handoff 06's enforce instructions marked as replaced by section 5 here

3. How it works

1. **The proxy** (`frontend/proxy.ts`) runs before each page request. It makes 16 random bytes with `crypto.getRandomValues` and encodes them as 24 base64 characters. That is 128 bits, so guessing is hopeless.
2. It puts the policy containing `'nonce-<value>'` on the **request** header `Content-Security-Policy`, which only Next.js sees. Next.js reads the nonce from there and adds `nonce="<value>"` to every `<script>` it writes: framework code, page chunks and its inline page data. The proxy overwrites any CSP header the caller sent, so a visitor cannot choose the nonce for their own page.
3. It sends the policy to the browser on the **response**, in Report-Only mode for now.
4. **Every page renders per request** (`await connection()` in the root layout), so every response gets a fresh nonce.
5. The proxy skips `/api/…`, `/auth/…`, `/health`, built files under `/_next/`, `/favicon.ico`, link prefetches, and `/python-worker.js`. The worker keeps its own enforced policy from `next.config.mjs`, unchanged: Pyodide only from `cdn.jsdelivr.net`.

The policy, and what changed from handoff 06

```
default-src 'self'; script-src 'self' 'nonce-<new every request>'; style-src 'self'; img-src 'self'; font-src 'self'; connect-src 'self'; worker-src 'self'; object-src 'none'; base-uri 'none'; form-action 'self'
```

- **Only change:** `script-src 'self' 'unsafe-inline'` became `script-src 'self' 'nonce-…'`. Everything else is exactly handoff 06's policy.
- **No `'strict-dynamic'`.** The Next.js guide adds it, but it makes the browser ignore `'self'` and trust any script that a trusted script inserts. We don't need it: every script Next.js loads later is a file from our own site, which `'self'` already allows. I tested without it (section 7). Leaving it out keeps the policy narrower.
- **Report-Only, next to the old policy.** One Next.js detail matters here: when the proxy sets `Content-Security-Policy-Report-Only`, it **replaces** the same header from `next.config.mjs` rather than adding to it. So the proxy now sends both policies in one header, separated by a comma, which browsers treat as two separate policies. The old one allows inline scripts, so it never reports anything the new one does not. It stays until you switch.

4. What I found while building it

- **Next.js's built-in 404 page breaks `style-src 'self'`.** It has an inline `<style>` tag and `style="…"` attributes with no nonce, under both handoff 06's policy and the new one. Handoff 06 never visited a missing page, so this was missed. With the policy enforced, the 404 page would have shown up unstyled. I added `app/not-found.tsx`: a heading, one sentence and a link home, using the app's own stylesheet. It is the only visible change.
- `next dev` **cannot run under a strict policy.** The development server puts styles in inline `<style>` tags and uses `eval`, and the old policy already reported both in development. So `next dev` always stays Report-Only, even after you set `ENFORCE = true`. Production builds (`npm run build`, Render, CI) do not need either.
- **Static rendering and nonces cannot be combined.** A static page is written once at build time, when there is no request and so no nonce. I tested this: with the pages left static, 26 of 30 tests fail. That is why every page now renders per request.

5. Switching to enforce mode (one line)

In `frontend/proxy.ts`, change:

```
const ENFORCE = false;
```

to:

```
const ENFORCE = true;
```

Pages then send this instead of the Report-Only header:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-<new every request>';
style-src 'self'; img-src 'self'; font-src 'self'; connect-src 'self'; worker-src 'self';
object-src 'none'; base-uri 'none'; form-action 'self'; frame-ancestors 'none'
```

Tested locally, the same way as handoff 06: I made exactly that change and rebuilt. - All 30 Playwright tests passed. - Against the real local API, with no mocks, a browser opened `/`, `/courses`, `/courses/py`, `/learn/hello` and a missing page, each with the enforced nonce policy. - On `/preview` it ran real Python (`A forest.\nA dragon!\n`), the real `/api/check` returned `passed`, and the Story spark badge appeared. There were no violations. - I then set it back to `false`.

Enforcing this policy also completes SEC-04, because it replaces handoff 06's enforce step.

6. Cost: pages lose static caching

Page	Before	After
`/` (home and welcome)	Static: built once, served from Next.js's cache	Rendered per request
`/courses`	Static	Rendered per request
`/preview`	Static	Rendered per request
404 page	Static	Rendered per request
`/courses/<course>`, `/learn/<lesson>`	Already rendered per request	Unchanged, plus the proxy
`/api/…`, `/auth/…`, `/python-worker.js`, `/_next/static/…`	—	Unchanged; the proxy does not run on them

Caches lost: - **Next.js's in-memory HTML cache** on our own Render instance. The live site shows `x-nextjs-cache: HIT` today. - **Shared caches:** pages used to send `Cache-Control: s-maxage=31536000`; they now send `private, no-cache, no-store`. That is required, because a cached page would hand everyone the same nonce, and a test checks for it. - **Cloudflare:** nothing lost. Live responses show `cf-cache-status: DYNAMIC`, so Cloudflare was not caching our pages anyway. - **Browsers:** some don't keep `no-store` pages in the back/forward cache, so the back button may reload the page instead of restoring it instantly.

Measured locally (production build, 400 requests, 10 at a time, Apple M5 Pro):

Page	Before	After
`/courses` (was static)	0.56 s (about 710 pages/s)	0.76 s (about 525 pages/s): about 0.5 ms more CPU per page
`/learn/hello` (already per request)	0.63 s	0.71 s: the proxy adds about 0.2 ms
Page size	5,724 bytes	6,105 bytes (the nonce attributes)
Server memory after the test	300 MB	357 MB

On our Render plan: - **Money:** the frontend is a Starter web service at a flat USD 7 a month (0.5 CPU, 512 MB). Render does not bill per request, so per-request rendering costs **nothing extra**. Bandwidth grows by about 400 bytes per page view. - **CPU:** Render's half-CPU is slower than this Mac, so expect a few milliseconds per page view rather than 0.5 ms. That is an estimate, not a measurement. For a handful of testers it is far below what the instance can do. - **Memory is the one thing to watch.** Locally the server used about 57 MB more after load, and Starter has 512 MB. Memory on a Mac is not the same as on Render, so section 10 asks you to read the frontend's Memory graph. If it ever gets close to 512 MB, the next plan up costs more. I would come back to you before suggesting that.

Smallest safe alternative, if this ever becomes a problem: Next.js has an experimental hash-based option (Subresource Integrity) that keeps pages static. It is marked experimental, and it covers script files, not

Next.js's changing inline page data, so it would not remove `'unsafe-inline'` by itself. I do not recommend it now. The nonce approach is the one Next.js documents, and it is not fragile here: three small files, no new dependencies, and a test fails if pages go static again.

7. Tests run (exact commands and results)

From the repository root, on macOS with Node 24:

Command	Result
`cd frontend && npx tsc --noEmit`	No errors
`cd frontend && npm run build`	Compiled successfully; every route shown as `ƒ` (rendered per request), plus `ƒ Proxy`
`cd frontend && CI=1 npm run test:e2e`	`30 passed` (Report-Only)
Same with `ENFORCE = true`	`30 passed`
Real local API, `ENFORCE = true`, browser script with no mocks	Section 5: Python ran, real check `passed`, badge shown, 0 violations
`cd backend && .venv/bin/python -m pytest -q`	Unchanged by this PR; CI runs it

New Playwright tests

Test	What it proves
`every page request gets a new nonce, and every script on the page carries it`	12 requests (6 pages, twice each, including the 404): 12 different nonces of 128 bits; the policy has no `'unsafe-inline'`; **every** `<script>` tag on the page carries that response's nonce; pages are sent with `no-store`
`every page loads with no CSP reports under the nonce policy`	`/`, `/courses`, `/courses/py`, `/learn/hello`, `/preview` and a missing page, each as a visitor, a student and a parent, with zero reports
`an injected inline script without the nonce is reported as blocked`	The page HTML is altered on the way in (like stored XSS) to add three scripts: one with no nonce, one with a guessed nonce, one with the real nonce. Exactly two `script-src-elem` reports, for the first two. In Report-Only mode those two still run; in enforce mode they do not run. The one with the real nonce always runs
`the lesson page runs Python with the security headers on` (updated)	The nonce policy is present with no `'unsafe-inline'`; the worker policy is unchanged; real Python runs and the challenge passes

All the existing tests, including the real Python runs for every lesson, also fail on any CSP report. So the whole suite checks the new policy.

Controls (proving the tests catch mistakes)

Deliberate break	Result
Nonce removed from the policy (`script-src 'self'` only)	29 failed
The same nonce on every request	2 failed: the nonce test and the header test
Pages left static (no `connection()`)	26 failed

Each break was undone afterwards.

8. Mocks and unverified behaviour

- **The Playwright tests mock the API**, as before. Python runs for real in the browser (Pyodide from the CDN). The enforce-mode check in section 5 used the real local API, with no mocks.
- **The injection test changes the HTML in the test browser.** It stands in for a real injection bug; there is no known injection bug in the app.
- **Speed and memory were measured on a Mac**, not on Render. The Render figures in section 6 are estimates.
- **Not verified:** Safari and Firefox (only Chromium was tested; nonces are standard CSP, supported by all current browsers), and the memory and response time on Render (section 10).

9. Questions for the owner

1. **SEC-09 before the admin inbox?** Still open from handoff 06: should enforcing this policy be a dependency of FB-06? I recommend yes.
2. **Report-Only for a week**, then the one-line switch: is that still the plan?

10. Live checks for you after merging

Merging redeploys the frontend only; the API is unchanged. Paste this first:

```
URL=https://python-academy-dji8.onrender.com
```

- [ ] **Deploy finished.** Render → **python-academy** shows the new deploy as live.
- [ ] **Headers.** Run `curl -sI $URL/courses | grep -i -E "content-security|cache-control"`. You should see:
- `content-security-policy: frame-ancestors 'none'`
- `content-security-policy-report-only:` with two policies separated by a comma, the second containing `script-src 'self' 'nonce-`
- `cache-control: private, no-cache, no-store, max-age=0, must-revalidate`

- [ ] **The nonce changes every time.** `for i in 1 2 3; do curl -s -D - -o /dev/null $URL/ | grep -io "nonce-[a-z0-9+/=]*"; done` prints three **different** values.
- [ ] **Every script carries it.** `curl -s $URL/learn/hello | grep -o '<script[^>]*>' | grep -vc 'nonce='` prints `0`.
- [ ] **Worker unchanged.** `curl -sI $URL/python-worker.js | grep -i content-security-policy:` shows `default-src 'none'; script-src 'unsafe-eval' https://cdn.jsdelivr.net; …`.
- [ ] **No reports during normal use.** Open DevTools → Console, then as a signed-in student visit the home page, `/courses`, a course page and a lesson. Run and check a challenge (badge and XP as before), and open "Change goal". Do the same as a parent, and visit a page that does not exist (for example `$URL/nothing-here`). The console should show **no** message containing `Content Security Policy`. Send me any you see, word for word.
- [ ] **An injected script would be blocked.** On any page, paste this into the DevTools console:

```
js const s=document.createElement('script');s.textContent='console.log("injected script ran")';document.head.append(s);
```

You should see a `[Report Only] Refused to execute inline script … 'nonce-…'` message, followed by `injected script ran`. In Report-Only mode it still runs; after the switch it will not. - [ ] **Memory and speed after a day.** Render → python-academy → Metrics: memory stays well below 512 MB, and response times look like they did before. Tell me the peak memory. - [ ] **After a week** of clean consoles, ask me for the `ENFORCE = true` PR (section 5).

11. Next concrete action

The owner reviews this PR, merges it, and runs section 10. Record the stage 7 live check (handoff 07 section 10) at the same time. After a week of clean consoles, a one-line PR enforces the nonce policy, which completes SEC-04 and SEC-09. Then FB-01 (decisions 14 and 17).